SKEIN ENGINE

VOLUME 10

NETWORKING AND STARCORE SERVER ARCHITECTURE

Transport, messaging, serialization, reliability, security hooks, headless runtime, replication foundations and future Linux server path.

DOCUMENT CODE	**SKEIN-IMP-002-V10**
VERSION	**0.1**
STATUS	**DRAFT CONSTRUCTION REFERENCE**
PLANNED PAGES	**145**
CHAPTERS	**13**
DATE	**30 July 2026**

IMPLEMENTATION MANUAL / PRINT SET

Current architecture source: SKEIN-SPEC-001 v0.4 and SKEIN-IMP-002-MASTER v0.1

10.01 Networking goals and authority model

Purpose and boundary: client/server

This page defines the purpose and boundary for Networking goals and authority model, with particular attention to client/server. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Client/server is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with trust boundaries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for client/server; distinguish required behavior from illustrative implementation detail.
- Keep trust boundaries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NetworkingGoalsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NetworkingGoalsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NetworkingGoalsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.01 and client/server.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.01 Networking goals and authority model

Architecture: trust boundaries

This page defines the architecture for Networking goals and authority model, with particular attention to trust boundaries. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Trust boundaries is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tick rates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for trust boundaries; distinguish required behavior from illustrative implementation detail.
- Keep tick rates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Networking", "NetworkingGoalsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.01 and trust boundaries.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.01 Networking goals and authority model

Data model: tick rates

This page defines the data model for Networking goals and authority model, with particular attention to tick rates. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Tick rates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with scope limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tick rates; distinguish required behavior from illustrative implementation detail.
- Keep scope limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NetworkingGoalsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NetworkingGoalsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NetworkingGoalsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.01 and tick rates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.01 Networking goals and authority model

Public API: scope limits

This page defines the public api for Networking goals and authority model, with particular attention to scope limits. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Scope limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with client/server is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for scope limits; distinguish required behavior from illustrative implementation detail.
- Keep client/server behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Networking", "NetworkingGoalsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.01 and scope limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.01 Networking goals and authority model

Ownership and lifetime: client/server

This page defines the ownership and lifetime for Networking goals and authority model, with particular attention to client/server. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Client/server is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with trust boundaries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for client/server; distinguish required behavior from illustrative implementation detail.
- Keep trust boundaries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NetworkingGoalsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NetworkingGoalsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NetworkingGoalsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.01 and client/server.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.01 Networking goals and authority model

Threading model: trust boundaries

This page defines the threading model for Networking goals and authority model, with particular attention to trust boundaries. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Trust boundaries is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tick rates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for trust boundaries; distinguish required behavior from illustrative implementation detail.
- Keep tick rates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Networking", "NetworkingGoalsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.01 and trust boundaries.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.01 Networking goals and authority model

Memory strategy: tick rates

This page defines the memory strategy for Networking goals and authority model, with particular attention to tick rates. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Tick rates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with scope limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tick rates; distinguish required behavior from illustrative implementation detail.
- Keep scope limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NetworkingGoalsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NetworkingGoalsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NetworkingGoalsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.01 and tick rates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.01 Networking goals and authority model

Failure handling: scope limits

This page defines the failure handling for Networking goals and authority model, with particular attention to scope limits. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Scope limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with client/server is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for scope limits; distinguish required behavior from illustrative implementation detail.
- Keep client/server behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Networking", "NetworkingGoalsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.01 and scope limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.01 Networking goals and authority model

Diagnostics: client/server

This page defines the diagnostics for Networking goals and authority model, with particular attention to client/server. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Client/server is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with trust boundaries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for client/server; distinguish required behavior from illustrative implementation detail.
- Keep trust boundaries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NetworkingGoalsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NetworkingGoalsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NetworkingGoalsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.01 and client/server.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.02 Socket platform abstraction

Purpose and boundary: UDP

This page defines the purpose and boundary for Socket platform abstraction, with particular attention to UDP. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Udp is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with TCP is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for UDP; distinguish required behavior from illustrative implementation detail.
- Keep TCP behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SocketPlatformAbstractionService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SocketPlatformAbstractionConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SocketPlatformAbstractionState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.02 and UDP.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.02 Socket platform abstraction

Architecture: TCP

This page defines the architecture for Socket platform abstraction, with particular attention to TCP. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Tcp is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with nonblocking I/O is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for TCP; distinguish required behavior from illustrative implementation detail.
- Keep nonblocking I/O behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Socket", "SocketPlatformAbstraction");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.02 and TCP.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.02 Socket platform abstraction

Data model: nonblocking I/O

This page defines the data model for Socket platform abstraction, with particular attention to nonblocking I/O. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Nonblocking i/o is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with addressing is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for nonblocking I/O; distinguish required behavior from illustrative implementation detail.
- Keep addressing behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SocketPlatformAbstractionService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SocketPlatformAbstractionConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SocketPlatformAbstractionState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.02 and nonblocking I/O.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.02 Socket platform abstraction

Public API: addressing

This page defines the public api for Socket platform abstraction, with particular attention to addressing. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Addressing is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with Windows/Linux boundary is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for addressing; distinguish required behavior from illustrative implementation detail.
- Keep Windows/Linux boundary behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Socket", "SocketPlatformAbstraction");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.02 and addressing.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.02 Socket platform abstraction

Ownership and lifetime: Windows/Linux boundary

This page defines the ownership and lifetime for Socket platform abstraction, with particular attention to Windows/Linux boundary. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Windows/linux boundary is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with UDP is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for Windows/Linux boundary; distinguish required behavior from illustrative implementation detail.
- Keep UDP behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SocketPlatformAbstractionService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SocketPlatformAbstractionConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SocketPlatformAbstractionState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.02 and Windows/Linux boundary.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.02 Socket platform abstraction

Threading model: UDP

This page defines the threading model for Socket platform abstraction, with particular attention to UDP. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Udp is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with TCP is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for UDP; distinguish required behavior from illustrative implementation detail.
- Keep TCP behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Socket", "SocketPlatformAbstraction");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.02 and UDP.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.02 Socket platform abstraction

Memory strategy: TCP

This page defines the memory strategy for Socket platform abstraction, with particular attention to TCP. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Tcp is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with nonblocking I/O is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for TCP; distinguish required behavior from illustrative implementation detail.
- Keep nonblocking I/O behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SocketPlatformAbstractionService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SocketPlatformAbstractionConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SocketPlatformAbstractionState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.02 and TCP.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.02 Socket platform abstraction

Failure handling: nonblocking I/O

This page defines the failure handling for Socket platform abstraction, with particular attention to nonblocking I/O. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Nonblocking i/o is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with addressing is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for nonblocking I/O; distinguish required behavior from illustrative implementation detail.
- Keep addressing behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Socket", "SocketPlatformAbstraction");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.02 and nonblocking I/O.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.02 Socket platform abstraction

Diagnostics: addressing

This page defines the diagnostics for Socket platform abstraction, with particular attention to addressing. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Addressing is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with Windows/Linux boundary is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for addressing; distinguish required behavior from illustrative implementation detail.
- Keep Windows/Linux boundary behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SocketPlatformAbstractionService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SocketPlatformAbstractionConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SocketPlatformAbstractionState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.02 and addressing.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.02 Socket platform abstraction

Implementation sequence: Windows/Linux boundary

This page defines the implementation sequence for Socket platform abstraction, with particular attention to Windows/Linux boundary. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Windows/linux boundary is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with UDP is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for Windows/Linux boundary; distinguish required behavior from illustrative implementation detail.
- Keep UDP behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Socket", "SocketPlatformAbstraction");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.02 and Windows/Linux boundary.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.02 Socket platform abstraction

Verification: UDP

This page defines the verification for Socket platform abstraction, with particular attention to UDP. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Udp is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with TCP is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for UDP; distinguish required behavior from illustrative implementation detail.
- Keep TCP behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SocketPlatformAbstractionService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SocketPlatformAbstractionConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SocketPlatformAbstractionState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.02 and UDP.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.03 Packet and message serialization

Purpose and boundary: bit streams

This page defines the purpose and boundary for Packet and message serialization, with particular attention to bit streams. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Bit streams is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with schemas is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for bit streams; distinguish required behavior from illustrative implementation detail.
- Keep schemas behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PacketAndMessageService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PacketAndMessageConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PacketAndMessageState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.03 and bit streams.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.03 Packet and message serialization

Architecture: schemas

This page defines the architecture for Packet and message serialization, with particular attention to schemas. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Schemas is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with versioning is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for schemas; distinguish required behavior from illustrative implementation detail.
- Keep versioning behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Packet", "PacketAndMessage");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.03 and schemas.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.03 Packet and message serialization

Data model: versioning

This page defines the data model for Packet and message serialization, with particular attention to versioning. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Versioning is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for versioning; distinguish required behavior from illustrative implementation detail.
- Keep limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PacketAndMessageService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PacketAndMessageConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PacketAndMessageState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.03 and versioning.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.03 Packet and message serialization

Public API: limits

This page defines the public api for Packet and message serialization, with particular attention to limits. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with validation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for limits; distinguish required behavior from illustrative implementation detail.
- Keep validation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Packet", "PacketAndMessage");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.03 and limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.03 Packet and message serialization

Ownership and lifetime: validation

This page defines the ownership and lifetime for Packet and message serialization, with particular attention to validation. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Validation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with bit streams is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for validation; distinguish required behavior from illustrative implementation detail.
- Keep bit streams behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PacketAndMessageService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PacketAndMessageConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PacketAndMessageState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.03 and validation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.03 Packet and message serialization

Threading model: bit streams

This page defines the threading model for Packet and message serialization, with particular attention to bit streams. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Bit streams is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with schemas is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for bit streams; distinguish required behavior from illustrative implementation detail.
- Keep schemas behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Packet", "PacketAndMessage");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.03 and bit streams.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.03 Packet and message serialization

Memory strategy: schemas

This page defines the memory strategy for Packet and message serialization, with particular attention to schemas. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Schemas is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with versioning is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for schemas; distinguish required behavior from illustrative implementation detail.
- Keep versioning behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PacketAndMessageService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PacketAndMessageConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PacketAndMessageState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.03 and schemas.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.03 Packet and message serialization

Failure handling: versioning

This page defines the failure handling for Packet and message serialization, with particular attention to versioning. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Versioning is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for versioning; distinguish required behavior from illustrative implementation detail.
- Keep limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Packet", "PacketAndMessage");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.03 and versioning.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.03 Packet and message serialization

Diagnostics: limits

This page defines the diagnostics for Packet and message serialization, with particular attention to limits. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with validation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for limits; distinguish required behavior from illustrative implementation detail.
- Keep validation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PacketAndMessageService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PacketAndMessageConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PacketAndMessageState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.03 and limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.03 Packet and message serialization

Implementation sequence: validation

This page defines the implementation sequence for Packet and message serialization, with particular attention to validation. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Validation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with bit streams is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for validation; distinguish required behavior from illustrative implementation detail.
- Keep bit streams behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Packet", "PacketAndMessage");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.03 and validation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.03 Packet and message serialization

Verification: bit streams

This page defines the verification for Packet and message serialization, with particular attention to bit streams. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Bit streams is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with schemas is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for bit streams; distinguish required behavior from illustrative implementation detail.
- Keep schemas behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PacketAndMessageService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PacketAndMessageConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PacketAndMessageState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.03 and bit streams.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.03 Packet and message serialization

Completion gate: schemas

This page defines the completion gate for Packet and message serialization, with particular attention to schemas. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Schemas is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with versioning is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for schemas; distinguish required behavior from illustrative implementation detail.
- Keep versioning behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Packet", "PacketAndMessage");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.03 and schemas.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.03 Packet and message serialization

Purpose and boundary: versioning

This page defines the purpose and boundary for Packet and message serialization, with particular attention to versioning. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Versioning is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for versioning; distinguish required behavior from illustrative implementation detail.
- Keep limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PacketAndMessageService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PacketAndMessageConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PacketAndMessageState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.03 and versioning.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.04 Connection lifecycle

Purpose and boundary: handshake

This page defines the purpose and boundary for Connection lifecycle, with particular attention to handshake. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Handshake is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with timeouts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for handshake; distinguish required behavior from illustrative implementation detail.
- Keep timeouts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ConnectionLifecycleService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ConnectionLifecycleConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ConnectionLifecycleState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.04 and handshake.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.04 Connection lifecycle

Architecture: timeouts

This page defines the architecture for Connection lifecycle, with particular attention to timeouts. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Timeouts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with disconnects is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for timeouts; distinguish required behavior from illustrative implementation detail.
- Keep disconnects behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Connection", "ConnectionLifecycle");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.04 and timeouts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.04 Connection lifecycle

Data model: disconnects

This page defines the data model for Connection lifecycle, with particular attention to disconnects. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Disconnects is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with reconnect is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for disconnects; distinguish required behavior from illustrative implementation detail.
- Keep reconnect behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ConnectionLifecycleService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ConnectionLifecycleConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ConnectionLifecycleState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.04 and disconnects.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.04 Connection lifecycle

Public API: reconnect

This page defines the public api for Connection lifecycle, with particular attention to reconnect. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Reconnect is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with diagnostics is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for reconnect; distinguish required behavior from illustrative implementation detail.
- Keep diagnostics behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Connection", "ConnectionLifecycle");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.04 and reconnect.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.04 Connection lifecycle

Ownership and lifetime: diagnostics

This page defines the ownership and lifetime for Connection lifecycle, with particular attention to diagnostics. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Diagnostics is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with handshake is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for diagnostics; distinguish required behavior from illustrative implementation detail.
- Keep handshake behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ConnectionLifecycleService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ConnectionLifecycleConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ConnectionLifecycleState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.04 and diagnostics.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.04 Connection lifecycle

Threading model: handshake

This page defines the threading model for Connection lifecycle, with particular attention to handshake. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Handshake is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with timeouts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for handshake; distinguish required behavior from illustrative implementation detail.
- Keep timeouts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Connection", "ConnectionLifecycle");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.04 and handshake.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.04 Connection lifecycle

Memory strategy: timeouts

This page defines the memory strategy for Connection lifecycle, with particular attention to timeouts. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Timeouts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with disconnects is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for timeouts; distinguish required behavior from illustrative implementation detail.
- Keep disconnects behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ConnectionLifecycleService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ConnectionLifecycleConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ConnectionLifecycleState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.04 and timeouts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.04 Connection lifecycle

Failure handling: disconnects

This page defines the failure handling for Connection lifecycle, with particular attention to disconnects. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Disconnects is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with reconnect is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for disconnects; distinguish required behavior from illustrative implementation detail.
- Keep reconnect behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Connection", "ConnectionLifecycle");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.04 and disconnects.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.04 Connection lifecycle

Diagnostics: reconnect

This page defines the diagnostics for Connection lifecycle, with particular attention to reconnect. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Reconnect is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with diagnostics is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for reconnect; distinguish required behavior from illustrative implementation detail.
- Keep diagnostics behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ConnectionLifecycleService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ConnectionLifecycleConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ConnectionLifecycleState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.04 and reconnect.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.04 Connection lifecycle

Implementation sequence: diagnostics

This page defines the implementation sequence for Connection lifecycle, with particular attention to diagnostics. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Diagnostics is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with handshake is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for diagnostics; distinguish required behavior from illustrative implementation detail.
- Keep handshake behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Connection", "ConnectionLifecycle");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.04 and diagnostics.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.04 Connection lifecycle

Verification: handshake

This page defines the verification for Connection lifecycle, with particular attention to handshake. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Handshake is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with timeouts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for handshake; distinguish required behavior from illustrative implementation detail.
- Keep timeouts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ConnectionLifecycleService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ConnectionLifecycleConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ConnectionLifecycleState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.04 and handshake.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.05 Reliability and ordering

Purpose and boundary: sequence numbers

This page defines the purpose and boundary for Reliability and ordering, with particular attention to sequence numbers. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Sequence numbers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with acks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for sequence numbers; distinguish required behavior from illustrative implementation detail.
- Keep acks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReliabilityAndOrderingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReliabilityAndOrderingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReliabilityAndOrderingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.05 and sequence numbers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.05 Reliability and ordering

Architecture: acks

This page defines the architecture for Reliability and ordering, with particular attention to acks. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Acks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with resends is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for acks; distinguish required behavior from illustrative implementation detail.
- Keep resends behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Reliability", "ReliabilityAndOrdering");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.05 and acks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.05 Reliability and ordering

Data model: resends

This page defines the data model for Reliability and ordering, with particular attention to resends. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Resends is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with channels is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for resends; distinguish required behavior from illustrative implementation detail.
- Keep channels behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReliabilityAndOrderingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReliabilityAndOrderingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReliabilityAndOrderingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.05 and resends.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.05 Reliability and ordering

Public API: channels

This page defines the public api for Reliability and ordering, with particular attention to channels. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Channels is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with congestion hooks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for channels; distinguish required behavior from illustrative implementation detail.
- Keep congestion hooks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Reliability", "ReliabilityAndOrdering");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.05 and channels.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.05 Reliability and ordering

Ownership and lifetime: congestion hooks

This page defines the ownership and lifetime for Reliability and ordering, with particular attention to congestion hooks. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Congestion hooks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with sequence numbers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for congestion hooks; distinguish required behavior from illustrative implementation detail.
- Keep sequence numbers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReliabilityAndOrderingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReliabilityAndOrderingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReliabilityAndOrderingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.05 and congestion hooks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.05 Reliability and ordering

Threading model: sequence numbers

This page defines the threading model for Reliability and ordering, with particular attention to sequence numbers. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Sequence numbers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with acks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for sequence numbers; distinguish required behavior from illustrative implementation detail.
- Keep acks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Reliability", "ReliabilityAndOrdering");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.05 and sequence numbers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.05 Reliability and ordering

Memory strategy: acks

This page defines the memory strategy for Reliability and ordering, with particular attention to acks. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Acks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with resends is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for acks; distinguish required behavior from illustrative implementation detail.
- Keep resends behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReliabilityAndOrderingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReliabilityAndOrderingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReliabilityAndOrderingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.05 and acks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.05 Reliability and ordering

Failure handling: resends

This page defines the failure handling for Reliability and ordering, with particular attention to resends. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Resends is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with channels is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for resends; distinguish required behavior from illustrative implementation detail.
- Keep channels behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Reliability", "ReliabilityAndOrdering");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.05 and resends.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.05 Reliability and ordering

Diagnostics: channels

This page defines the diagnostics for Reliability and ordering, with particular attention to channels. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Channels is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with congestion hooks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for channels; distinguish required behavior from illustrative implementation detail.
- Keep congestion hooks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReliabilityAndOrderingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReliabilityAndOrderingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReliabilityAndOrderingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.05 and channels.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.05 Reliability and ordering

Implementation sequence: congestion hooks

This page defines the implementation sequence for Reliability and ordering, with particular attention to congestion hooks. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Congestion hooks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with sequence numbers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for congestion hooks; distinguish required behavior from illustrative implementation detail.
- Keep sequence numbers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Reliability", "ReliabilityAndOrdering");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.05 and congestion hooks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.05 Reliability and ordering

Verification: sequence numbers

This page defines the verification for Reliability and ordering, with particular attention to sequence numbers. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Sequence numbers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with acks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for sequence numbers; distinguish required behavior from illustrative implementation detail.
- Keep acks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReliabilityAndOrderingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReliabilityAndOrderingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReliabilityAndOrderingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.05 and sequence numbers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.05 Reliability and ordering

Completion gate: acks

This page defines the completion gate for Reliability and ordering, with particular attention to acks. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Acks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with resends is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for acks; distinguish required behavior from illustrative implementation detail.
- Keep resends behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Reliability", "ReliabilityAndOrdering");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.05 and acks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.05 Reliability and ordering

Purpose and boundary: resends

This page defines the purpose and boundary for Reliability and ordering, with particular attention to resends. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Resends is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with channels is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for resends; distinguish required behavior from illustrative implementation detail.
- Keep channels behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReliabilityAndOrderingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReliabilityAndOrderingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReliabilityAndOrderingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.05 and resends.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.06 Time synchronization and clocks

Purpose and boundary: RTT

This page defines the purpose and boundary for Time synchronization and clocks, with particular attention to RTT. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Rtt is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with offset is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for RTT; distinguish required behavior from illustrative implementation detail.
- Keep offset behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class TimeSynchronizationAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const TimeSynchronizationAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    TimeSynchronizationAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.06 and RTT.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.06 Time synchronization and clocks

Architecture: offset

This page defines the architecture for Time synchronization and clocks, with particular attention to offset. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Offset is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with drift is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for offset; distinguish required behavior from illustrative implementation detail.
- Keep drift behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Time", "TimeSynchronizationAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.06 and offset.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.06 Time synchronization and clocks

Data model: drift

This page defines the data model for Time synchronization and clocks, with particular attention to drift. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Drift is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with server tick mapping is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for drift; distinguish required behavior from illustrative implementation detail.
- Keep server tick mapping behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class TimeSynchronizationAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const TimeSynchronizationAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    TimeSynchronizationAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.06 and drift.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.06 Time synchronization and clocks

Public API: server tick mapping

This page defines the public api for Time synchronization and clocks, with particular attention to server tick mapping. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Server tick mapping is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with RTT is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for server tick mapping; distinguish required behavior from illustrative implementation detail.
- Keep RTT behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Time", "TimeSynchronizationAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.06 and server tick mapping.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.06 Time synchronization and clocks

Ownership and lifetime: RTT

This page defines the ownership and lifetime for Time synchronization and clocks, with particular attention to RTT. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Rtt is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with offset is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for RTT; distinguish required behavior from illustrative implementation detail.
- Keep offset behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class TimeSynchronizationAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const TimeSynchronizationAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    TimeSynchronizationAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.06 and RTT.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.06 Time synchronization and clocks

Threading model: offset

This page defines the threading model for Time synchronization and clocks, with particular attention to offset. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Offset is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with drift is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for offset; distinguish required behavior from illustrative implementation detail.
- Keep drift behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Time", "TimeSynchronizationAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.06 and offset.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.06 Time synchronization and clocks

Memory strategy: drift

This page defines the memory strategy for Time synchronization and clocks, with particular attention to drift. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Drift is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with server tick mapping is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for drift; distinguish required behavior from illustrative implementation detail.
- Keep server tick mapping behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class TimeSynchronizationAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const TimeSynchronizationAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    TimeSynchronizationAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.06 and drift.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.06 Time synchronization and clocks

Failure handling: server tick mapping

This page defines the failure handling for Time synchronization and clocks, with particular attention to server tick mapping. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Server tick mapping is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with RTT is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for server tick mapping; distinguish required behavior from illustrative implementation detail.
- Keep RTT behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Time", "TimeSynchronizationAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.06 and server tick mapping.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.06 Time synchronization and clocks

Diagnostics: RTT

This page defines the diagnostics for Time synchronization and clocks, with particular attention to RTT. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Rtt is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with offset is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for RTT; distinguish required behavior from illustrative implementation detail.
- Keep offset behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class TimeSynchronizationAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const TimeSynchronizationAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    TimeSynchronizationAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.06 and RTT.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.07 Security and authentication hooks

Purpose and boundary: encryption boundary

This page defines the purpose and boundary for Security and authentication hooks, with particular attention to encryption boundary. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Encryption boundary is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tokens is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for encryption boundary; distinguish required behavior from illustrative implementation detail.
- Keep tokens behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SecurityAndAuthenticationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SecurityAndAuthenticationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SecurityAndAuthenticationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.07 and encryption boundary.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.07 Security and authentication hooks

Architecture: tokens

This page defines the architecture for Security and authentication hooks, with particular attention to tokens. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Tokens is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with rate limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tokens; distinguish required behavior from illustrative implementation detail.
- Keep rate limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Security", "SecurityAndAuthentication");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.07 and tokens.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.07 Security and authentication hooks

Data model: rate limits

This page defines the data model for Security and authentication hooks, with particular attention to rate limits. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Rate limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with malformed traffic is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for rate limits; distinguish required behavior from illustrative implementation detail.
- Keep malformed traffic behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SecurityAndAuthenticationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SecurityAndAuthenticationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SecurityAndAuthenticationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.07 and rate limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.07 Security and authentication hooks

Public API: malformed traffic

This page defines the public api for Security and authentication hooks, with particular attention to malformed traffic. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Malformed traffic is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with encryption boundary is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for malformed traffic; distinguish required behavior from illustrative implementation detail.
- Keep encryption boundary behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Security", "SecurityAndAuthentication");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.07 and malformed traffic.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.07 Security and authentication hooks

Ownership and lifetime: encryption boundary

This page defines the ownership and lifetime for Security and authentication hooks, with particular attention to encryption boundary. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Encryption boundary is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tokens is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for encryption boundary; distinguish required behavior from illustrative implementation detail.
- Keep tokens behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SecurityAndAuthenticationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SecurityAndAuthenticationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SecurityAndAuthenticationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.07 and encryption boundary.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.07 Security and authentication hooks

Threading model: tokens

This page defines the threading model for Security and authentication hooks, with particular attention to tokens. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Tokens is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with rate limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tokens; distinguish required behavior from illustrative implementation detail.
- Keep rate limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Security", "SecurityAndAuthentication");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.07 and tokens.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.07 Security and authentication hooks

Memory strategy: rate limits

This page defines the memory strategy for Security and authentication hooks, with particular attention to rate limits. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Rate limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with malformed traffic is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for rate limits; distinguish required behavior from illustrative implementation detail.
- Keep malformed traffic behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SecurityAndAuthenticationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SecurityAndAuthenticationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SecurityAndAuthenticationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.07 and rate limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.07 Security and authentication hooks

Failure handling: malformed traffic

This page defines the failure handling for Security and authentication hooks, with particular attention to malformed traffic. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Malformed traffic is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with encryption boundary is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for malformed traffic; distinguish required behavior from illustrative implementation detail.
- Keep encryption boundary behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Security", "SecurityAndAuthentication");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.07 and malformed traffic.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.07 Security and authentication hooks

Diagnostics: encryption boundary

This page defines the diagnostics for Security and authentication hooks, with particular attention to encryption boundary. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Encryption boundary is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tokens is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for encryption boundary; distinguish required behavior from illustrative implementation detail.
- Keep tokens behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SecurityAndAuthenticationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SecurityAndAuthenticationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SecurityAndAuthenticationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.07 and encryption boundary.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.07 Security and authentication hooks

Implementation sequence: tokens

This page defines the implementation sequence for Security and authentication hooks, with particular attention to tokens. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Tokens is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with rate limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tokens; distinguish required behavior from illustrative implementation detail.
- Keep rate limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Security", "SecurityAndAuthentication");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.07 and tokens.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.07 Security and authentication hooks

Verification: rate limits

This page defines the verification for Security and authentication hooks, with particular attention to rate limits. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Rate limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with malformed traffic is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for rate limits; distinguish required behavior from illustrative implementation detail.
- Keep malformed traffic behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SecurityAndAuthenticationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SecurityAndAuthenticationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SecurityAndAuthenticationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.07 and rate limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.08 Headless runtime architecture

Purpose and boundary: no-render startup

This page defines the purpose and boundary for Headless runtime architecture, with particular attention to no-render startup. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

No-render startup is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with service lifecycle is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for no-render startup; distinguish required behavior from illustrative implementation detail.
- Keep service lifecycle behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class HeadlessRuntimeArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const HeadlessRuntimeArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    HeadlessRuntimeArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.08 and no-render startup.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.08 Headless runtime architecture

Architecture: service lifecycle

This page defines the architecture for Headless runtime architecture, with particular attention to service lifecycle. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Service lifecycle is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with console is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for service lifecycle; distinguish required behavior from illustrative implementation detail.
- Keep console behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Headless", "HeadlessRuntimeArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.08 and service lifecycle.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.08 Headless runtime architecture

Data model: console

This page defines the data model for Headless runtime architecture, with particular attention to console. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Console is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with configuration is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for console; distinguish required behavior from illustrative implementation detail.
- Keep configuration behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class HeadlessRuntimeArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const HeadlessRuntimeArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    HeadlessRuntimeArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.08 and console.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.08 Headless runtime architecture

Public API: configuration

This page defines the public api for Headless runtime architecture, with particular attention to configuration. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Configuration is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with signals is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for configuration; distinguish required behavior from illustrative implementation detail.
- Keep signals behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Headless", "HeadlessRuntimeArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.08 and configuration.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.08 Headless runtime architecture

Ownership and lifetime: signals

This page defines the ownership and lifetime for Headless runtime architecture, with particular attention to signals. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Signals is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with no-render startup is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for signals; distinguish required behavior from illustrative implementation detail.
- Keep no-render startup behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class HeadlessRuntimeArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const HeadlessRuntimeArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    HeadlessRuntimeArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.08 and signals.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.08 Headless runtime architecture

Threading model: no-render startup

This page defines the threading model for Headless runtime architecture, with particular attention to no-render startup. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

No-render startup is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with service lifecycle is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for no-render startup; distinguish required behavior from illustrative implementation detail.
- Keep service lifecycle behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Headless", "HeadlessRuntimeArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.08 and no-render startup.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.08 Headless runtime architecture

Memory strategy: service lifecycle

This page defines the memory strategy for Headless runtime architecture, with particular attention to service lifecycle. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Service lifecycle is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with console is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for service lifecycle; distinguish required behavior from illustrative implementation detail.
- Keep console behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class HeadlessRuntimeArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const HeadlessRuntimeArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    HeadlessRuntimeArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.08 and service lifecycle.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.08 Headless runtime architecture

Failure handling: console

This page defines the failure handling for Headless runtime architecture, with particular attention to console. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Console is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with configuration is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for console; distinguish required behavior from illustrative implementation detail.
- Keep configuration behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Headless", "HeadlessRuntimeArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.08 and console.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.08 Headless runtime architecture

Diagnostics: configuration

This page defines the diagnostics for Headless runtime architecture, with particular attention to configuration. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Configuration is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with signals is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for configuration; distinguish required behavior from illustrative implementation detail.
- Keep signals behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class HeadlessRuntimeArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const HeadlessRuntimeArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    HeadlessRuntimeArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.08 and configuration.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.08 Headless runtime architecture

Implementation sequence: signals

This page defines the implementation sequence for Headless runtime architecture, with particular attention to signals. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Signals is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with no-render startup is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for signals; distinguish required behavior from illustrative implementation detail.
- Keep no-render startup behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Headless", "HeadlessRuntimeArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.08 and signals.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.08 Headless runtime architecture

Verification: no-render startup

This page defines the verification for Headless runtime architecture, with particular attention to no-render startup. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

No-render startup is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with service lifecycle is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for no-render startup; distinguish required behavior from illustrative implementation detail.
- Keep service lifecycle behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class HeadlessRuntimeArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const HeadlessRuntimeArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    HeadlessRuntimeArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.08 and no-render startup.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.08 Headless runtime architecture

Completion gate: service lifecycle

This page defines the completion gate for Headless runtime architecture, with particular attention to service lifecycle. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Service lifecycle is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with console is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for service lifecycle; distinguish required behavior from illustrative implementation detail.
- Keep console behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Headless", "HeadlessRuntimeArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.08 and service lifecycle.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.08 Headless runtime architecture

Purpose and boundary: console

This page defines the purpose and boundary for Headless runtime architecture, with particular attention to console. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Console is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with configuration is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for console; distinguish required behavior from illustrative implementation detail.
- Keep configuration behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class HeadlessRuntimeArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const HeadlessRuntimeArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    HeadlessRuntimeArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.08 and console.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.09 Starcore server plugin boundary

Purpose and boundary: services

This page defines the purpose and boundary for Starcore server plugin boundary, with particular attention to services. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Services is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with world ownership is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for services; distinguish required behavior from illustrative implementation detail.
- Keep world ownership behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class StarcoreServerPluginService final
{
public:
    [[nodiscard]] Result<void> Initialise(const StarcoreServerPluginConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    StarcoreServerPluginState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.09 and services.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.09 Starcore server plugin boundary

Architecture: world ownership

This page defines the architecture for Starcore server plugin boundary, with particular attention to world ownership. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

World ownership is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with persistence hooks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for world ownership; distinguish required behavior from illustrative implementation detail.
- Keep persistence hooks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Starcore", "StarcoreServerPlugin");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.09 and world ownership.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.09 Starcore server plugin boundary

Data model: persistence hooks

This page defines the data model for Starcore server plugin boundary, with particular attention to persistence hooks. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Persistence hooks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with observability is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for persistence hooks; distinguish required behavior from illustrative implementation detail.
- Keep observability behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class StarcoreServerPluginService final
{
public:
    [[nodiscard]] Result<void> Initialise(const StarcoreServerPluginConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    StarcoreServerPluginState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.09 and persistence hooks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.09 Starcore server plugin boundary

Public API: observability

This page defines the public api for Starcore server plugin boundary, with particular attention to observability. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Observability is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with services is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for observability; distinguish required behavior from illustrative implementation detail.
- Keep services behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Starcore", "StarcoreServerPlugin");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.09 and observability.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.09 Starcore server plugin boundary

Ownership and lifetime: services

This page defines the ownership and lifetime for Starcore server plugin boundary, with particular attention to services. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Services is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with world ownership is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for services; distinguish required behavior from illustrative implementation detail.
- Keep world ownership behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class StarcoreServerPluginService final
{
public:
    [[nodiscard]] Result<void> Initialise(const StarcoreServerPluginConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    StarcoreServerPluginState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.09 and services.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.09 Starcore server plugin boundary

Threading model: world ownership

This page defines the threading model for Starcore server plugin boundary, with particular attention to world ownership. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

World ownership is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with persistence hooks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for world ownership; distinguish required behavior from illustrative implementation detail.
- Keep persistence hooks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Starcore", "StarcoreServerPlugin");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.09 and world ownership.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.09 Starcore server plugin boundary

Memory strategy: persistence hooks

This page defines the memory strategy for Starcore server plugin boundary, with particular attention to persistence hooks. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Persistence hooks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with observability is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for persistence hooks; distinguish required behavior from illustrative implementation detail.
- Keep observability behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class StarcoreServerPluginService final
{
public:
    [[nodiscard]] Result<void> Initialise(const StarcoreServerPluginConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    StarcoreServerPluginState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.09 and persistence hooks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.09 Starcore server plugin boundary

Failure handling: observability

This page defines the failure handling for Starcore server plugin boundary, with particular attention to observability. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Observability is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with services is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for observability; distinguish required behavior from illustrative implementation detail.
- Keep services behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Starcore", "StarcoreServerPlugin");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.09 and observability.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.09 Starcore server plugin boundary

Diagnostics: services

This page defines the diagnostics for Starcore server plugin boundary, with particular attention to services. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Services is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with world ownership is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for services; distinguish required behavior from illustrative implementation detail.
- Keep world ownership behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class StarcoreServerPluginService final
{
public:
    [[nodiscard]] Result<void> Initialise(const StarcoreServerPluginConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    StarcoreServerPluginState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.09 and services.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.09 Starcore server plugin boundary

Implementation sequence: world ownership

This page defines the implementation sequence for Starcore server plugin boundary, with particular attention to world ownership. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

World ownership is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with persistence hooks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for world ownership; distinguish required behavior from illustrative implementation detail.
- Keep persistence hooks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Starcore", "StarcoreServerPlugin");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.09 and world ownership.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.09 Starcore server plugin boundary

Verification: persistence hooks

This page defines the verification for Starcore server plugin boundary, with particular attention to persistence hooks. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Persistence hooks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with observability is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for persistence hooks; distinguish required behavior from illustrative implementation detail.
- Keep observability behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class StarcoreServerPluginService final
{
public:
    [[nodiscard]] Result<void> Initialise(const StarcoreServerPluginConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    StarcoreServerPluginState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.09 and persistence hooks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.09 Starcore server plugin boundary

Completion gate: observability

This page defines the completion gate for Starcore server plugin boundary, with particular attention to observability. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Observability is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with services is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for observability; distinguish required behavior from illustrative implementation detail.
- Keep services behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Starcore", "StarcoreServerPlugin");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.09 and observability.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.09 Starcore server plugin boundary

Purpose and boundary: services

This page defines the purpose and boundary for Starcore server plugin boundary, with particular attention to services. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Services is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with world ownership is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for services; distinguish required behavior from illustrative implementation detail.
- Keep world ownership behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class StarcoreServerPluginService final
{
public:
    [[nodiscard]] Result<void> Initialise(const StarcoreServerPluginConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    StarcoreServerPluginState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.09 and services.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Purpose and boundary: entity IDs

This page defines the purpose and boundary for Replication foundations, with particular attention to entity IDs. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Entity ids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with snapshots is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for entity IDs; distinguish required behavior from illustrative implementation detail.
- Keep snapshots behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReplicationFoundationsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReplicationFoundationsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReplicationFoundationsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and entity IDs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Architecture: snapshots

This page defines the architecture for Replication foundations, with particular attention to snapshots. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Snapshots is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with interest management is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for snapshots; distinguish required behavior from illustrative implementation detail.
- Keep interest management behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Replication", "ReplicationFoundations");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and snapshots.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Data model: interest management

This page defines the data model for Replication foundations, with particular attention to interest management. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Interest management is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with property descriptors is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for interest management; distinguish required behavior from illustrative implementation detail.
- Keep property descriptors behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReplicationFoundationsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReplicationFoundationsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReplicationFoundationsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and interest management.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Public API: property descriptors

This page defines the public api for Replication foundations, with particular attention to property descriptors. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Property descriptors is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with entity IDs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for property descriptors; distinguish required behavior from illustrative implementation detail.
- Keep entity IDs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Replication", "ReplicationFoundations");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and property descriptors.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Ownership and lifetime: entity IDs

This page defines the ownership and lifetime for Replication foundations, with particular attention to entity IDs. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Entity ids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with snapshots is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for entity IDs; distinguish required behavior from illustrative implementation detail.
- Keep snapshots behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReplicationFoundationsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReplicationFoundationsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReplicationFoundationsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and entity IDs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Threading model: snapshots

This page defines the threading model for Replication foundations, with particular attention to snapshots. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Snapshots is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with interest management is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for snapshots; distinguish required behavior from illustrative implementation detail.
- Keep interest management behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Replication", "ReplicationFoundations");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and snapshots.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Memory strategy: interest management

This page defines the memory strategy for Replication foundations, with particular attention to interest management. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Interest management is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with property descriptors is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for interest management; distinguish required behavior from illustrative implementation detail.
- Keep property descriptors behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReplicationFoundationsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReplicationFoundationsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReplicationFoundationsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and interest management.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Failure handling: property descriptors

This page defines the failure handling for Replication foundations, with particular attention to property descriptors. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Property descriptors is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with entity IDs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for property descriptors; distinguish required behavior from illustrative implementation detail.
- Keep entity IDs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Replication", "ReplicationFoundations");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and property descriptors.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Diagnostics: entity IDs

This page defines the diagnostics for Replication foundations, with particular attention to entity IDs. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Entity ids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with snapshots is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for entity IDs; distinguish required behavior from illustrative implementation detail.
- Keep snapshots behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReplicationFoundationsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReplicationFoundationsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReplicationFoundationsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and entity IDs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Implementation sequence: snapshots

This page defines the implementation sequence for Replication foundations, with particular attention to snapshots. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Snapshots is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with interest management is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for snapshots; distinguish required behavior from illustrative implementation detail.
- Keep interest management behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Replication", "ReplicationFoundations");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and snapshots.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Verification: interest management

This page defines the verification for Replication foundations, with particular attention to interest management. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Interest management is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with property descriptors is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for interest management; distinguish required behavior from illustrative implementation detail.
- Keep property descriptors behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReplicationFoundationsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReplicationFoundationsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReplicationFoundationsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and interest management.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Completion gate: property descriptors

This page defines the completion gate for Replication foundations, with particular attention to property descriptors. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Property descriptors is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with entity IDs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for property descriptors; distinguish required behavior from illustrative implementation detail.
- Keep entity IDs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Replication", "ReplicationFoundations");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and property descriptors.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Purpose and boundary: entity IDs

This page defines the purpose and boundary for Replication foundations, with particular attention to entity IDs. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Entity ids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with snapshots is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for entity IDs; distinguish required behavior from illustrative implementation detail.
- Keep snapshots behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReplicationFoundationsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReplicationFoundationsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReplicationFoundationsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and entity IDs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Architecture: snapshots

This page defines the architecture for Replication foundations, with particular attention to snapshots. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Snapshots is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with interest management is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for snapshots; distinguish required behavior from illustrative implementation detail.
- Keep interest management behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Replication", "ReplicationFoundations");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and snapshots.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.10 Replication foundations

Data model: interest management

This page defines the data model for Replication foundations, with particular attention to interest management. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Interest management is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with property descriptors is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for interest management; distinguish required behavior from illustrative implementation detail.
- Keep property descriptors behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class ReplicationFoundationsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const ReplicationFoundationsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    ReplicationFoundationsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.10 and interest management.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.11 Prediction and reconciliation plan

Purpose and boundary: input commands

This page defines the purpose and boundary for Prediction and reconciliation plan, with particular attention to input commands. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Input commands is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with client prediction is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for input commands; distinguish required behavior from illustrative implementation detail.
- Keep client prediction behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PredictionAndReconciliationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PredictionAndReconciliationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PredictionAndReconciliationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.11 and input commands.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.11 Prediction and reconciliation plan

Architecture: client prediction

This page defines the architecture for Prediction and reconciliation plan, with particular attention to client prediction. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Client prediction is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with corrections is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for client prediction; distinguish required behavior from illustrative implementation detail.
- Keep corrections behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Prediction", "PredictionAndReconciliation");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.11 and client prediction.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.11 Prediction and reconciliation plan

Data model: corrections

This page defines the data model for Prediction and reconciliation plan, with particular attention to corrections. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Corrections is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with physics boundary is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for corrections; distinguish required behavior from illustrative implementation detail.
- Keep physics boundary behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PredictionAndReconciliationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PredictionAndReconciliationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PredictionAndReconciliationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.11 and corrections.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.11 Prediction and reconciliation plan

Public API: physics boundary

This page defines the public api for Prediction and reconciliation plan, with particular attention to physics boundary. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Physics boundary is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with input commands is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for physics boundary; distinguish required behavior from illustrative implementation detail.
- Keep input commands behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Prediction", "PredictionAndReconciliation");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.11 and physics boundary.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.11 Prediction and reconciliation plan

Ownership and lifetime: input commands

This page defines the ownership and lifetime for Prediction and reconciliation plan, with particular attention to input commands. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Input commands is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with client prediction is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for input commands; distinguish required behavior from illustrative implementation detail.
- Keep client prediction behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PredictionAndReconciliationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PredictionAndReconciliationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PredictionAndReconciliationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.11 and input commands.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.11 Prediction and reconciliation plan

Threading model: client prediction

This page defines the threading model for Prediction and reconciliation plan, with particular attention to client prediction. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Client prediction is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with corrections is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for client prediction; distinguish required behavior from illustrative implementation detail.
- Keep corrections behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Prediction", "PredictionAndReconciliation");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.11 and client prediction.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.11 Prediction and reconciliation plan

Memory strategy: corrections

This page defines the memory strategy for Prediction and reconciliation plan, with particular attention to corrections. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Corrections is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with physics boundary is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for corrections; distinguish required behavior from illustrative implementation detail.
- Keep physics boundary behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PredictionAndReconciliationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PredictionAndReconciliationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PredictionAndReconciliationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.11 and corrections.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.11 Prediction and reconciliation plan

Failure handling: physics boundary

This page defines the failure handling for Prediction and reconciliation plan, with particular attention to physics boundary. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Physics boundary is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with input commands is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for physics boundary; distinguish required behavior from illustrative implementation detail.
- Keep input commands behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Prediction", "PredictionAndReconciliation");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.11 and physics boundary.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.11 Prediction and reconciliation plan

Diagnostics: input commands

This page defines the diagnostics for Prediction and reconciliation plan, with particular attention to input commands. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Input commands is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with client prediction is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for input commands; distinguish required behavior from illustrative implementation detail.
- Keep client prediction behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PredictionAndReconciliationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PredictionAndReconciliationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PredictionAndReconciliationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.11 and input commands.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.12 Network Insights integration

Purpose and boundary: packets

This page defines the purpose and boundary for Network Insights integration, with particular attention to packets. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Packets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with messages is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for packets; distinguish required behavior from illustrative implementation detail.
- Keep messages behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NetworkInsightsIntegrationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NetworkInsightsIntegrationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NetworkInsightsIntegrationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.12 and packets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.12 Network Insights integration

Architecture: messages

This page defines the architecture for Network Insights integration, with particular attention to messages. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Messages is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with bandwidth is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for messages; distinguish required behavior from illustrative implementation detail.
- Keep bandwidth behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Network", "NetworkInsightsIntegration");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.12 and messages.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.12 Network Insights integration

Data model: bandwidth

This page defines the data model for Network Insights integration, with particular attention to bandwidth. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Bandwidth is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with latency is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for bandwidth; distinguish required behavior from illustrative implementation detail.
- Keep latency behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NetworkInsightsIntegrationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NetworkInsightsIntegrationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NetworkInsightsIntegrationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.12 and bandwidth.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.12 Network Insights integration

Public API: latency

This page defines the public api for Network Insights integration, with particular attention to latency. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Latency is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with loss is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for latency; distinguish required behavior from illustrative implementation detail.
- Keep loss behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Network", "NetworkInsightsIntegration");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.12 and latency.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.12 Network Insights integration

Ownership and lifetime: loss

This page defines the ownership and lifetime for Network Insights integration, with particular attention to loss. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Loss is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with packets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for loss; distinguish required behavior from illustrative implementation detail.
- Keep packets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NetworkInsightsIntegrationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NetworkInsightsIntegrationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NetworkInsightsIntegrationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.12 and loss.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.12 Network Insights integration

Threading model: packets

This page defines the threading model for Network Insights integration, with particular attention to packets. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Packets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with messages is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for packets; distinguish required behavior from illustrative implementation detail.
- Keep messages behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Network", "NetworkInsightsIntegration");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.12 and packets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.12 Network Insights integration

Memory strategy: messages

This page defines the memory strategy for Network Insights integration, with particular attention to messages. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Messages is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with bandwidth is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for messages; distinguish required behavior from illustrative implementation detail.
- Keep bandwidth behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NetworkInsightsIntegrationService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NetworkInsightsIntegrationConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NetworkInsightsIntegrationState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.12 and messages.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.12 Network Insights integration

Failure handling: bandwidth

This page defines the failure handling for Network Insights integration, with particular attention to bandwidth. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Bandwidth is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with latency is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for bandwidth; distinguish required behavior from illustrative implementation detail.
- Keep latency behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Network", "NetworkInsightsIntegration");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.12 and bandwidth.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.13 Load, soak and failure testing

Purpose and boundary: bots

This page defines the purpose and boundary for Load, soak and failure testing, with particular attention to bots. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Bots is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with packet loss is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for bots; distinguish required behavior from illustrative implementation detail.
- Keep packet loss behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LoadSoakAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LoadSoakAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LoadSoakAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.13 and bots.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.13 Load, soak and failure testing

Architecture: packet loss

This page defines the architecture for Load, soak and failure testing, with particular attention to packet loss. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Packet loss is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with long runs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for packet loss; distinguish required behavior from illustrative implementation detail.
- Keep long runs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Load", "LoadSoakAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.13 and packet loss.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.13 Load, soak and failure testing

Data model: long runs

This page defines the data model for Load, soak and failure testing, with particular attention to long runs. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Long runs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with resource limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for long runs; distinguish required behavior from illustrative implementation detail.
- Keep resource limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LoadSoakAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LoadSoakAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LoadSoakAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.13 and long runs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.13 Load, soak and failure testing

Public API: resource limits

This page defines the public api for Load, soak and failure testing, with particular attention to resource limits. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Resource limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with recovery is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for resource limits; distinguish required behavior from illustrative implementation detail.
- Keep recovery behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Load", "LoadSoakAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.13 and resource limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.13 Load, soak and failure testing

Ownership and lifetime: recovery

This page defines the ownership and lifetime for Load, soak and failure testing, with particular attention to recovery. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Recovery is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with bots is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for recovery; distinguish required behavior from illustrative implementation detail.
- Keep bots behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LoadSoakAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LoadSoakAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LoadSoakAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.13 and recovery.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.13 Load, soak and failure testing

Threading model: bots

This page defines the threading model for Load, soak and failure testing, with particular attention to bots. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Bots is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with packet loss is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for bots; distinguish required behavior from illustrative implementation detail.
- Keep packet loss behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Load", "LoadSoakAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.13 and bots.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.13 Load, soak and failure testing

Memory strategy: packet loss

This page defines the memory strategy for Load, soak and failure testing, with particular attention to packet loss. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Packet loss is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with long runs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for packet loss; distinguish required behavior from illustrative implementation detail.
- Keep long runs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LoadSoakAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LoadSoakAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LoadSoakAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.13 and packet loss.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.13 Load, soak and failure testing

Failure handling: long runs

This page defines the failure handling for Load, soak and failure testing, with particular attention to long runs. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Long runs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with resource limits is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for long runs; distinguish required behavior from illustrative implementation detail.
- Keep resource limits behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Load", "LoadSoakAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 10.13 and long runs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

10.13 Load, soak and failure testing

Diagnostics: resource limits

This page defines the diagnostics for Load, soak and failure testing, with particular attention to resource limits. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Resource limits is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with recovery is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for resource limits; distinguish required behavior from illustrative implementation detail.
- Keep recovery behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LoadSoakAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LoadSoakAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LoadSoakAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 10.13 and resource limits.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.